



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
PROYECTO DE CARRERA: INGENIERÍA EN INFORMÁTICA
CÁTEDRA: INGENIERÍA DE SOFTWARE I

SISTEMA DE GESTIÓN Y CONTROL OPERATIVO PARA ESTACIONES DE SERVICIO (CROSS-STATION)

DOCENTE:
DUBRASKA
ROKA

NOMBRE:
WULLIBER YEPEZ
V-32.130.926

1. INTRODUCCIÓN

La administración y operación diaria de las estaciones de servicio de combustible representan un desafío logístico de alta criticidad, donde convergen variables de alta volatilidad, márgenes reducidos y estrictos requerimientos de control volumétrico y de seguridad. Históricamente, muchas estaciones operan mediante conciliaciones manuales o sistemas descentralizados que dificultan la detección oportuna de anomalías, exponiendo el negocio a desabastecimientos imprevistos, cuellos de botella durante las operaciones de descarga de cisternas, y pérdidas silenciosas ocasionadas por microfugas o descalibraciones en los surtidores.

El presente documento formaliza la etapa de análisis, especificación de requerimientos y diseño estructural para el desarrollo de una solución de software moderna orientada a la gestión integral de estaciones de servicio. Diseñado bajo el marco de trabajo ágil Scrum, el sistema busca automatizar el registro transaccional en pista, garantizar la trazabilidad de inventarios físicos y lógicos en tiempo real, incorporar algoritmos de Inteligencia Artificial para la predicción de demanda y reabastecimiento óptimo, y orquestar la consolidación y distribución automatizada de reportes gerenciales.

A continuación, se exponen los resultados del proceso de elicitación, la especificación de los requerimientos funcionales y no funcionales, la formulación de las historias de usuario orientadas al valor de negocio, y la definición de la estructura lógica de los diagramas de casos de uso para los distintos actores del sistema.

2. DESARROLLO

2.1. Proceso de Elicitación de Requerimientos

En consonancia con el marco ágil seleccionado (Scrum), el proceso de elicitación no se concibió como una recopilación pasiva de especificaciones en un documento estático, sino como un ejercicio interactivo y continuo de inmersión en el dominio del problema y descomposición funcional de los cuellos de botella que impactan la rentabilidad y operatividad de la estación de servicio.

El proceso iterativo se estructuró a través de tres fases fundamentales:

1. **Análisis del Dominio de Negocio:** Se identificaron y caracterizaron las variables críticas que condicionan la estabilidad operativa, tales como el riesgo de desabastecimiento, los cuellos de botella en la descarga de cisternas y las pérdidas acumuladas por microfugas o evaporación no controlada.
2. Definición y Modelado de Actores: Se establecieron límites de responsabilidad claros entre los dos perfiles principales del sistema: el Operador de Pista (encargado de la carga transaccional, registro de ventas y cierres de turno) y el Gerente (responsable del análisis estratégico, auditoría volumétrica mediante aforos físicos y toma de decisiones).
3. Traducción a Épicas Ágiles: Los problemas identificados fueron transformados en módulos funcionales incrementales y de alto valor de negocio: Control de Accesos, Operativa de Pista, Gestión de Fugas / Conciliación, e Inteligencia Artificial y Automatización.

2.2. Requerimientos del Sistema

2.2.1. Requerimientos Funcionales (RF)

- **RF01:** El sistema debe permitir el registro de nuevos operadores y su posterior aprobación, rechazo o suspensión por parte del gerente.

- RF02: El sistema debe registrar las salidas de combustible (Gasolina/Diesel) por transacción única, turno o día completo, restando el volumen del inventario lógico.
- RF03: El sistema debe permitir al gerente registrar aforos físicos de los tanques para ejecutar la conciliación volumétrica.
- RF04: El sistema debe calcular la diferencia entre el inventario teórico y el físico, emitiendo alertas si la desviación supera el umbral de tolerancia establecido (fuga/descalibración).
- RF05: El sistema debe integrar un módulo de IA que reciba el histórico de ventas y la capacidad máxima instalada para proyectar y sugerir la fecha y volumen del próximo pedido.
- RF06: El sistema debe permitir configurar la hora de ejecución para la consolidación de datos y el envío automático de reportes (diarios y semanales) vía correo electrónico.

2.2.2. Requerimientos No Funcionales (RNF)

- **RNF01 (Arquitectura Base del Frontend):** El frontend debe estar desarrollado nativamente en Next.js utilizando Tailwind CSS para una interfaz de usuario responsiva, moderna y optimizada, alojado y desplegado de forma continua en la infraestructura de Vercel.
- **RNF02 (Persistencia y Motor de Datos):** La base de datos debe ser relacional bajo el motor PostgreSQL, gestionada y aprovisionada a través de la plataforma en la nube Supabase.
- **RNF03 (Seguridad y Control de Acceso a Nivel de Fila):** Se deben implementar políticas RLS (Row Level Security) nativas en PostgreSQL para asegurar que cada rol (Operador o Gerente) acceda únicamente a los registros y vistas explícitamente autorizados.
- **RNF04 (Orquestación y Automatización Externa):** Los flujos de consolidación nocturna, cálculo en lote y mensajería transaccional/reportes programados deben orquestarse de manera desacoplada mediante la herramienta de automatización n8n.

2.3. Historias de Usuario

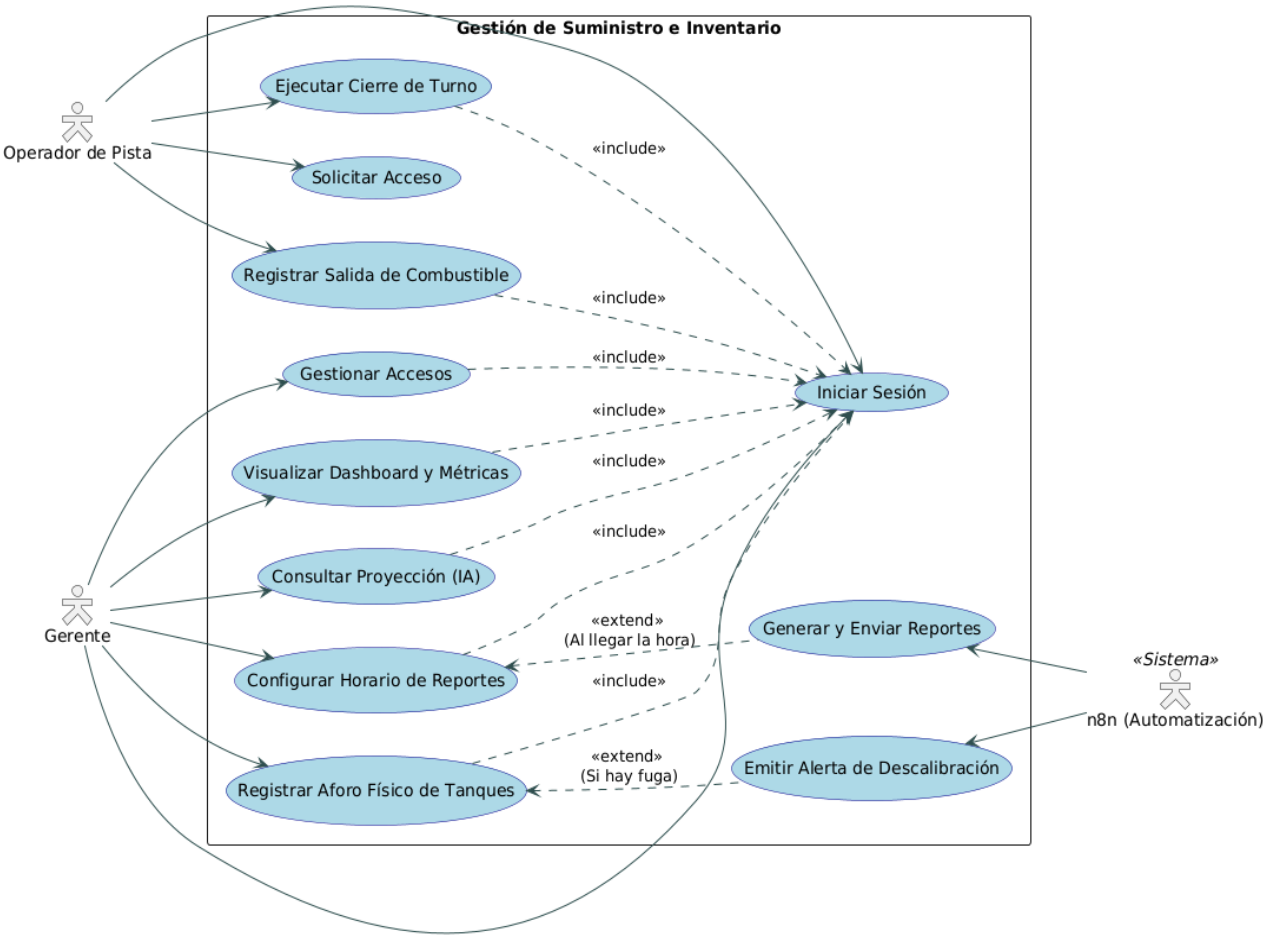
Se estructuran manteniendo el formato estándar de valor:

ID	Épica / Módulo	Rol	Historia de Usuario
HU-01	Control de Accesos	Operador	Como nuevo empleado, quiero registrar mis datos para solicitar acceso al sistema de gestión de la pista.
HU-02	Control de Accesos	Gerente	Como gerente, quiero un panel de solicitudes pendientes para aprobar o rechazar el acceso de operadores al sistema.
HU-03	Operativa de Pista	Operador	Como operador aprobado, quiero registrar ventas para mantener el inventario lógico de los tanques actualizado en tiempo real.
HU-04	Operativa de Pista	Operador	Como operador, quiero registrar mi cierre de turno para delimitar formalmente mi responsabilidad operativa.
HU-05	Métricas Visuales	Gerente	Como gerente, quiero visualizar gráficos del historial de consumo mensual para analizar tendencias de demanda y estacionalidad.
HU-06	Gestión de Fugas	Gerente	Como gerente, quiero registrar el aforo físico de los tanques para que el sistema calcule desviaciones y alerte de descalibraciones o pérdidas.
HU-07	Inteligencia Artificial	Gerente	Como gerente, quiero consultar al asistente de IA basado en mis ventas y

			capacidad para que me sugiera cuándo y cuánto combustible pedir.
HU-08	Automatización	Gerente	Como gerente, quiero configurar la consolidación periódica de datos para recibir reportes diarios y semanales automáticamente en mi correo.

2.4. Diagramas de Caso de Uso (Estructura Lógica)

La estructura lógica de casos de uso modela las interacciones entre los actores del sistema y los límites funcionales de la aplicación, agrupados en dos subsistemas principales



2.5. Integración de la Inteligencia Artificial

Para resolver el problema de "Desabastecimiento por mala proyección", se implementó el modelo `nvidia/llama3-chatqa-1.5-70b`, consumido a través de la API REST de NVIDIA (<https://integrate.api.nvidia.com/v1>). La IA actúa como un motor de inferencia que cruza la capacidad volumétrica instalada con el ritmo de consumo diario, proyectando los días de autonomía restantes y sugiriendo el volumen exacto de litros a pedir a la planta distribuidora.

Prompts utilizados:

- **System Prompt:** "Eres un experto analista en logística de hidrocarburos. Tu objetivo es prever el desabastecimiento. Responde strictly con un objeto JSON válido que contenga las claves: 'dias_autonomia' (número), 'volumen_sugerido_litros' (número) y 'justificacion' (texto breve)."

Consideraciones y Skills aplicados:

- **Zero-Shot Prompting con Restricción de Salida:** Se obligó al modelo a devolver exclusivamente un objeto JSON, permitiendo que el frontend de Next.js procese la respuesta programáticamente sin necesidad de limpiar texto conversacional innecesario.
- **Gestión Eficiente de Tokens:** Se fijó el parámetro `max_tokens: 300` y `temperature: 0.1` para garantizar respuestas cortas, deterministas y altamente precisas, reduciendo drásticamente el consumo de tokens y los costos operativos de la API.

Paso a paso del proceso:

1. El gerente presiona el botón "Proyectar Pedido" en el Dashboard.
2. El frontend recopila el nivel físico actual y calcula el promedio de ventas de los últimos 7 días desde la base de datos Supabase.
3. El backend construye el prompt inyectando estas variables y ejecuta un POST a la API de NVIDIA.

La respuesta JSON se renderiza en una tarjeta de alerta en la interfaz del gerente.

2.6. Prototipo UI | UX



2.7. Arquitectura general de la aplicación

El sistema se estructura bajo un modelo desacoplado basado en servicios en la nube:

- **Capa de Presentación (Frontend):** Desarrollada con Next.js y estilizada con Tailwind CSS, desplegada en la infraestructura de Vercel. Gestiona el enrutamiento protegido y las interfaces diferenciadas para Gerente y Operador.
- **Capa de Lógica y Persistencia (BaaS):** Centralizada en Supabase, que provee el motor de autenticación, políticas de seguridad RLS (Row Level Security) y la exposición de una API REST generada dinámicamente a partir del modelo de datos.
- **Capa de Inferencia Inteligente:** Consumo asíncrono del modelo LLaMA3 (vía API REST de NVIDIA) para analizar el ritmo de vaciado de los tanques y emitir sugerencias de pedidos.
- **Capa de Automatización:** Orquestada de forma independiente a través de flujos en n8n para procesar reportes en segundo plano sin bloquear el hilo principal de la aplicación.

2.8. Arquitectura de la Base Datos

Se implementó un modelo relacional normalizado sobre PostgreSQL.

- **Separación de Entidades:** El esquema divide lógicamente el estado físico continuo (tanques, aforos) de los eventos transaccionales discretos (ventas, cierres_turno).
- **Integridad Numérica:** Se aplicó el tipo de dato DECIMAL(10, 2) para todas las columnas de volumen, eliminando el riesgo de descuadres por matemática de punto flotante.
- **Trazabilidad y Seguridad:** Toda salida de combustible exige integridad referencial, enlazando el ID del tanque, el UUID del operador (desde auth.users) y el turno activo.

2.9. Arquitectura de Automatizaciones

El flujo asíncrono de reportes de cierre se construye en n8n mediante la siguiente topología de nodos:

- **Disparador (Webhook):** Escucha peticiones HTTP POST, activadas mediante una acción en el panel de Next.js o un proceso Cron.
- **Extracción (Nodo PostgreSQL):** Al recibir la señal, el nodo ejecuta una consulta de agregación (SUM) directa a la base de datos para compilar los litros totales vendidos en el periodo.

Transformación y Entrega (Nodo Email): La data cruda se inyecta en una plantilla HTML y se despacha al correo de gerencia utilizando un servidor SMTP.

3.0 Link del repositorio (Github) del proyecto

<https://github.com/AveRyCarNey/Cross-Station>

4.0 Captures de las pantallas de la aplicación

